

Welcome to the FRED Tutorial Guide!

Copyright © 1997, Thinkage Ltd.

FRED, the Friendly Editor, is a program which allows you to type in text, edit it, and write it out to a file. Your text can be the source for a computer program, data for a program, or ordinary English for a document like this one (this tutorial was prepared using FRED). Beginning users are most likely to use FRED interactively, i.e. by typing in text and commands from a terminal.

This tutorial is designed for people who have never used FRED and who may not be very familiar with computing in general. We only assume that you know how to sign on to the TSS system and that you have a vague notion of what a computer file is.

In this tutorial guide, we will try to tell you everything you need to know in order to do simple editing jobs. The odds are though, that you'll have questions we don't answer here. Some of the answers may be in the FRED Reference Manual, also available on the system. Other users who are more familiar with FRED may be able to help you with your problems too. Sometimes the best way to find things out though is just to experiment: if you wonder whether something will work, type in a few lines to play with and try it! New users should realize that nothing disastrous will happen, even if you make a mistake or two -- FRED takes care of you. (FRED also has a technique for letting you undo your mistakes; we'll talk about that later on.)

We recommend that you do all the exercises in this tutorial as you work your way through, and that you experiment with FRED as you go along too. If you happen to make a typing mistake, don't worry. FRED is designed to help you correct your mistakes. The worst thing that could happen is that FRED could give you an error message because he doesn't understand what you want him to do. In most cases, all you have to do is type in the line correctly and FRED will be happy again. We will say more about error messages very shortly.

Getting Started:

We will assume that you are signed on to the TSS system and that it has just prompted you with a "*" to find out what you want to do. To get started with FRED, type

```
fred<cr>
```

where <cr> is "return" or "carriage return". (We will use this symbol for "carriage return" occasionally throughout this tutorial, but most of the time we will just omit it when it's obvious a carriage return is needed.) When FRED is ready to go, he will answer by typing

```
Fred
```

on a new line.

FRED is now waiting for you to give him a command. Most FRED commands are just one letter long, although there are some with two letters. The letters can be either upper or lower case, but we will use lower case in this tutorial. It is sometimes possible to type in a number of commands on a single line; however, beginning users should generally type one command per line to avoid making things too complicated. When you are ready to give FRED something to do, type in the line and push the carriage return key -- the <cr> is what lets FRED know there's work to be done.

The first problem a new user is apt to have is getting text into the computer. Perhaps you want to type in the first draft of a report. Later on, you will use FRED to modify and edit this first draft into something more polished. For now though, we'll just talk about putting in your first rough copy.

If you were working on a normal typewriter, you would roll in a blank piece of paper and begin typing your report. When FRED first starts, it's very much like that blank piece of paper -- there is no text present yet. What you have is an empty "buffer". A buffer is the equivalent of a piece of scratch paper, a space inside the

computer where FRED keeps the text you're working on. You can type things into a buffer, change some of the things that are there, and finally file the whole thing away when it's in the shape you want it.

To tell FRED that you are ready to type things into your empty buffer, you type

a

immediately followed by a carriage return. The "a" stands for "append" or "add" and tells FRED that the next lines you type will be text to be stored in your buffer. FRED will answer by typing the character

>

which says FRED is ready to take a line of your text.

So to begin typing text into the empty buffer, we just type something like this:

```
a
>Mary had little lamb
>Its fleece was white as snow,
>And everywhere that Mary went,
>The lamb was sure to go.
```

FRED prompts you with a ">" every time he's ready for another line, and you type in the text you want.

Sooner or later, you will be finished typing in your text. At that point, you have to tell FRED that you're done with text input. You do this by typing

\F

immediately followed by a carriage return when FRED uses ">" to prompt for a new line. FRED will not put the "\F" into the buffer with the rest of your text -- you just type the "\F" to tell FRED that you're finished with text.

After he has seen the "\F", FRED will wait for you to give him something new to do. He will not prompt you with a ">", since this prompting is done only for text input and FRED knows you are no longer typing in text. If you want to see what you have typed in so far, type

*p

followed by a carriage return. The "p" means "print" and the "*" tells FRED that you want to print everything. FRED will oblige by typing

```
Mary had a little lamb
Its fleece was white as snow,
And everywhere that Mary went,
The lamb was sure to go.
```

Notice that the "a" and "\F" aren't there, and neither are the ">" characters. The only things in the buffer are the lines of text you typed in.

There's one thing we should point out about "*p": you can only use it when you aren't typing in lines of text. If you type "*p" while putting in lines of text, FRED will just assume the "*p" is part of your text, not a command.

Now if you want to type in more lines, just type "a" again and FRED will prompt with a ">" for more text.

A Few Important Words:

You are now in a position to type in a complete report using "a" to begin text input and "\F" to end it. You can also print what you've typed in by using "*p". You can type in practically anything you want, and as much as you want (a buffer can hold billions of lines).

The only thing you have to be careful with is the backslash character "\". This character is very special in FRED, both when you're giving FRED commands and when you're typing in text. We'll explain more about this in a few pages. Right now, all we'll say about it is that if you want to enter a backslash into your text lines, you have to type TWO backslashes with no space in between (\\). Try it.

```
a
>hello\\goodbye
>\F
```

If you now type "*"p" you'll find that FRED has converted the two backslashes into the single one you wanted.

```
hello\goodbye
```

Sooner or later you're bound to make a mistake. If you make a mistake typing in your lines of text, you'll just have to live with it for a while. Eventually we'll show you how easy it is to correct such mistakes using FRED (a lot easier than correcting mistakes on paper, once you get the hang of it).

The other kind of mistake you can make is giving FRED a command he doesn't understand. For example, try typing "\F" when you aren't putting in text. FRED will beep and print

```
? unknown command
```

on your terminal. The reason this happens is that "\F" is used to tell FRED you've finished typing in text. If you weren't typing in text, FRED doesn't understand why you typed a "\F".

Whenever you make an error in FRED, FRED will beep and issue an error message beginning with a question mark. Most of the time, it's fairly easy to figure out why FRED has given you an error by checking what you just typed in. If the reason for an error isn't obvious, you can have FRED give a bit more of an explanation by typing

```
y
```

(which stands for "Why?"). FRED will answer your question with a short description of why he gave you an error. If it's still not clear why the error happened, you should ask someone who is more familiar with FRED to help you out.

Pressing the BREAK or ATTN key while working with FRED is not quite an error, but it often gives unwanted results. BREAK or ATTN tells FRED to stop whatever he is doing. If you are typing in text, pressing BREAK or ATTN tells FRED to stop taking in text, just as "\F" does. However, if you press BREAK or ATTN by accident in the middle of a text line, not only does FRED stop taking in text but he loses the line you were typing in too. If this happens, just type "a" to begin appending lines again and start with the line that the BREAK or ATTN interrupted.

If FRED is printing your text following a "*"p" command, pressing BREAK or ATTN will stop the printing in the middle of things. This is sometimes what you want to do if the listing is long and you don't want to read everything.

When FRED is interrupted by BREAK or ATTN, he usually types a "?" just in case you don't realize what you've done. The Y command doesn't bother to explain this, since the "?" without a number is only issued when you push the BREAK or ATTN key by accident or on purpose.

Writing Text to a File:

We now know how to start putting in text ("a"), how to stop putting in text ("\F"), and how to print what text we have ("*p"). The next thing we want to know is how to write our text into a file. This is very important. We mentioned that your buffer could be thought of as *scratch* paper. This scratch paper is lost when you leave FRED (in the same way that papers left lying around on a desk tend to get lost too). If you want to preserve all the text you've just typed in, you have to write it out to a *file*. This is much like making a clean copy of your scratch paper and then filing it away someplace safe.

To write out everything that's in your buffer, you use the write command

```
w /myfile
```

(make sure there's a space between the "w" and the "/"). This instruction will write everything in your buffer into a permanent file named "/myfile". Of course, "/myfile" is just an example, and you can give your file any name you want.

If "/myfile" existed before you issued your write command, the contents of your buffer write over what was previously in "/myfile". Because of this, the previous contents of "/myfile" are lost. If "/myfile" did not exist before your write, FRED will create a permanent file with that name and put the contents of your buffer in the file.

It may take a short while for the write operation to finish, especially if you are writing out a large amount of text. When the writing is finished, FRED will display statistics for the writing operation. These have the form

```
12,345,6789 b(name) user/cat/file
```

You needn't concern yourself with these statistics when you're just starting out, but for your information, the first number is the number of blocks of disk storage your file is taking up, the second is the number of text lines FRED wrote out, and the third is the number of characters in those lines of text. After that comes the name of your current buffer and the name of the file where your text was written. If FRED has created a permanent file for you, a "p" will be printed out at the end of the line.

You must wait for the statistics to print out before you give FRED another command like "a" or "*p". Until those statistics are printed, FRED is busy writing and won't pay attention to anything you type in.

Perhaps you are wondering why you had to start the name of your file with a slash "/" in the write command. The reason is that the slash tells FRED you want your text in a permanent file. If there is no slash, FRED will look for a file called "myfile" and if he can't find a file with that name, he will create a temporary file. A temporary file is generally not what you want, since temporary files disappear when you sign off the terminal. To be able to keep your text from one editing session to the next, you have to keep your text in a permanent file.

If you happen to forget the slash with your file name and FRED creates a temporary file, a "t" will be printed out after the three numbers that tell about the write. If you see such a "t", you should probably give FRED another write command with the slash in the right place to make sure you get a permanent file instead of a temporary one.

A write operation simply copies the contents of your buffer into the file you name. The buffer itself is not changed at all. Thus you can append a few lines of text, stop text input with a "\F", write your buffer out, and then begin appending again right where you left off. Remember that you must stop your text input with a "\F" before you do your write; otherwise, FRED will just assume that "w /myfile" is one of your lines of text.

It's a good idea to write your buffer out to your file fairly often. If the system happens to crash (i.e. die) while you're working, you'll lose everything you've typed in since the last time you did a write.

Leaving FRED:

When you are through with FRED, you should write out the text you've been working on using the W command. Once you've done this, you can quit by typing

```
q
```

(which naturally means Quit). FRED will go away and in the process your buffer will be wiped clean. When the system is ready, it will print a "*" on your terminal to show that it is ready for something new. You can either sign off at this point by saying "logoff" or go on to different work. You might want to say "list myfile" when the system gives you the "*". The "list" system command will list the contents of "myfile" so you can see the text which you have written into it.

If you happen to type "q" BEFORE you have written out your file, don't worry. FRED will give you the warning

```
? questionable buffers
```

FRED gives you this warning because he assumes you really don't want to quit before saving your work. If you get this warning, write out your buffer and then say "q" again. This time FRED will let you leave in peace.

Exercise 1:

Enter FRED by replying "fred" when the system prompts you with its "*". Create some text and print it using

```
a
>...text...
>\F
*p
```

Write your text out to a permanent file using the W command. Quit FRED using the Q command.

Reading:

We now come to your second session with FRED. You sign on to the TSS system, you answer with "fred" when the system prompts with "*", and FRED types "Fred" to tell you he's ready for business. Once again, your buffer is completely empty as it was the first time you signed on. Naturally you'd like to get hold of all the text you typed in the last time you were working with FRED. Since you wrote all your text onto a permanent file, you can get the text back with the command

```
r /myfile
```

(where R stands for Read). This command gets the contents of "/myfile" and puts them into your empty buffer. If you now type

```
*p
```

FRED will type out

```
Mary had a little lamb
Its fleece was white as snow,
And everywhere that Mary went,
The lamb was sure to go.
```

(or whatever text you wrote into your file). You can now type "a" and start appending lines to the end of your text if you so desire.

There are one or two things that could go wrong when you try the read command. When you type "r /myfile", FRED assumes you want to read the contents of "myfile" into an empty buffer. If your buffer ISN'T empty, FRED will not let the read take place (since it would read the file in on top of the text already in the buffer and mess things up). If you try a read when your buffer already contains text, FRED will give you a

```
? buffer not empty
```

error message. If this happens, the only thing you can do at the moment is write out your buffer with "w", quit FRED with "q", and then come back to FRED again by answering "fred" to the system's "*" prompt. This will give you a fresh new empty buffer, and you can then read in your file. In a few pages, we'll tell you about better ways to clear a buffer for reading, but for now leaving FRED in this way will have to do.

Another thing that could happen when you try a read is that FRED will answer with

```
? incorrect cat/file description
```

This means that for some reason FRED couldn't find the file you wanted. Quite possibly, you have just made a typing mistake when you were typing in the file's name. If that's the case, just give the R command again, typing the file name correctly this time, and FRED will read in the file you want. If this isn't the case, the "y" command may be able to tell you what's wrong with your read.

After FRED has successfully read in a file, it prints the same kind of statistics that it prints when a file is written out. You might have to wait a little while after you give FRED your R command before these statistics are printed out, especially if FRED is reading in a large file. However, you must wait for the statistics to print before you give FRED another command, since FRED ignores anything you say while he's reading.

Line Numbers:

Every line in your buffer has a number. Line 1 is naturally the first line in the buffer, line 2 is the second and so on. To print a given line, all you have to do is type the line number followed by a "p" for print. For example, to print the first line of your text you would type

```
1p
```

and FRED would respond with

```
Mary had a little lamb
```

You can print more than one line by specifying the first line and last line you want printed, as in

```
1,3p
```

This command would give you

```
Mary had a little lamb  
Its fleece was white as snow,  
And everywhere that Mary went,
```

In other words, "1,3p" prints lines 1 to 3 inclusive.

Very often, you will want to have a look at the last line of text in the buffer to see what you've typed so far. Because it's annoying to have to keep track of the exact number of the last line of text, FRED allows you to use the character "\$" as a shorthand for the last line in the buffer. Thus

```
$p
```

is a command to print the last line of the buffer. The command

```
1,$p
```

will print all the lines from the first line in the buffer to the last. Obviously then, "1,\$p" is the same thing as "*p". A command like

```
3,$p
```

will print the current buffer from line 3 to the end.

Another valid form of line number is used in

```
$_-1,$p
```

This command prints the last two lines of the buffer, since "\$-1" is the second last line in the buffer and "\$" is the last.

Printing a line or group of lines is one of the most common things any user does when using FRED. Because of this, FRED lets you cut down on your typing by omitting the "p" in print commands. For example,

1,3

is the same as "1,3p". In the same way,

\$

prints the last line of the buffer.

Exercise 2:

Sign on to FRED and read in the file you created in Exercise 1. Print the file using "*p" or "1,\$p". Append a few more lines of text, then put in the "\F" to end text input.

Now experiment with the p command. Print several ranges of lines and see the many ways you can use the "\$" as in "\$-1", "\$-2" etc. Try printing line 0 (you'll get an error). See what happens when you use a line number so big it's past the end of the buffer. Also see what happens when you try something like

4,2p

where you tell FRED to begin printing at one line and end printing at an earlier line. Lastly see what happens when you give a P command more than two numbers as in

1,2,3p

Since the P command needs at most two line numbers, FRED will give you an error if you give three or more. You should also practice printing lines without typing the "p" as in

1,\$-1

What happens when you just type "*"?

The Current Line:

We have already learned that each line in the buffer has a line number and that we can print a line by typing its line number followed by a "p". If our buffer contains the verse of "Mary Had a Little Lamb" that we've been using in our examples,

2p

will print

Its fleece was white as snow,

What happens now if we simply type "p" with no line number?

p
Its fleece was white as snow,

FRED prints us the same line again!

The reason for this is very simple. Out of all the lines in our buffer, FRED always "keeps his finger" on one line. The line that FRED's finger is on at any given time is called "the current line". Just think of the current line as the line where you are currently working. For example, the command "2p" works on line 2 by printing it, and therefore the command makes line 2 the current line. If you then say "p" without a line number, FRED assumes you want him to print the current line. If you keep on typing in "p", FRED will keep on printing line 2 for you because that is the current line. If you type in "3p", the current line will be changed to line 3 (i.e. FRED moves his finger from line 2 to line 3). Consequently, typing "p" will now print line 3, the new current line.

Just as the shorthand symbol for the last line in the buffer is "\$", the shorthand symbol for the current line is

.

(pronounced "dot"). Dot is a line number in the same way "\$" is.

```
.,.+1p
```

will print the current line and the line after it.

```
.,$p
```

will print the lines from the current line to the end of the buffer.

As we have said, the current line of the buffer is the line where you are currently working. If you give FRED a command which changes where you are working, FRED automatically changes the current line too. For example, try

```
2,$p
```

and then try a "p" by itself. The single "p" will print the last line in the buffer. Why? "2,\$p" prints the lines from line 2 to the end of the buffer. FRED will set the current line to the last line of the buffer, since that is the last line printed (and therefore the last line where you were working). "p" by itself prints the current line and now the current line is the last line in the buffer.

It's a good idea to experiment a bit to make sure you understand about the current line. Remember that a "p" by itself will print the current line so you can see where you are. Another helpful command is

```
=
```

When you type this, FRED will answer by giving the line number of the current line. For example, you can find out the line number of the last line by typing

```
$  
=
```

The "\$" prints the last line and makes that the current line. The "=" tells the line number of the current line (the line number of the last line in the buffer). You can save some time by typing

```
$=
```

which tells you the line number of "\$". In this case, the current line is not changed to "\$", and "\$" is not printed. If our buffer contains "Mary had a little lamb", FRED would give "4" as the line number of the last line.

One last comment while we are talking about printing lines. A single carriage return with nothing else on the line will print the line after the current line. It's a handy equivalent for ".+1p" and saves a good deal of typing. Try it.

More About Line Numbers and the Current Line:

We have spoken a lot about the use of line numbers with the P command. We can also use line numbers with many of the other commands we have talked about. For example, try the command

```
1a  
>Hickory dickory dock  
>The mouse ran up the clock.  
>\F
```

The first question we might ask is what the current line is now. If we type "p" to print the current line, FRED prints

```
The mouse ran up the clock.
```

This is perfectly reasonable. The current line is the line where we are currently working, and this line of text is the last line we did anything with (we typed it in).

If we now print the contents of our buffer using `"*p"` or `"1,$p"`, FRED gives us

```
Mary had a little lamb
Hickory dickory dock
The mouse ran up the clock.
Its fleece was white as snow,
And everywhere that Mary went,
The lamb was sure to go.
```

The two lines we typed in were appended after line 1!

We see that we can append text lines after any line in the buffer by typing the line number followed by the `"a"` for append. If we want to append lines at the end of the buffer, we could say

```
$a
```

and then begin typing in our text.

What happens if we just say `"a"` without specifying a line number? Just as FRED assumes you want to print the current line when you say `"p"` without a line number, FRED assumes you want to append lines after the current line when you just say `"a"`. (If you look at the work we did at the very beginning of this tutorial, you will see that whenever we said `"a"` we were either at the end of the buffer or in an empty buffer. At that point we wanted to avoid appending text in the midst of lines that were already there.)

Using the `A` command in the middle of the buffer, you can append text after any line. You can even add things before the first line in the buffer by saying `"0a"` (appending things after line zero is the same thing as inserting them before line 1). This is very convenient when you want to add something in the middle of what you've already typed in. Just make sure your current line is in the right place, and append away.

We can also use line addresses with the `R` command. If we say

```
$r /myfile
```

the contents of `"myfile"` will be appended after the last line of the buffer. You will remember that FRED gives an error when you just say

```
r /myfile
```

and the buffer isn't empty. The reason is that FRED doesn't know where you want the contents of `"myfile"` put in the buffer and so he avoids problems by not doing anything. If you specify a line number, FRED is quite happy to read in the file and append its contents after the line you specify.

Exercise 3:

Enter FRED and append a few lines of text. Write the lines out to a file, then read the file back in by specifying a line number before the `"r"` in the `R` command. Print the contents of your buffer to see what has happened. Try a few commands like

```
.-1p
.r /file
.+1a
```

and see what happens.

Changing Text:

To review what we have done so far, we can read and write files (using `R` and `W`), we can append text into our buffer (using `A`), and we can print what is in our buffer (using `P`). We are now at the point where we want

to edit and modify the contents of our buffer.

The simplest command for doing this is the Change command, denoted by the letter "c". If, for example, we type in

```
3c
```

FRED will respond with ">" indicating he is ready to take lines of text input. FRED will take the text you type in and use it to replace line 3. For example, if you type

```
3c
>Row, row, row your boat
>Gently down the stream
>\F
```

the buffer will now contain

```
Mary had a little lamb
Hickory dickory dock
Row, row, row your boat
Gently down the stream
Its fleece was white as snow,
And everywhere that Mary went,
The lamb was sure to go.
```

You see that you can put in any number of lines to replace the given line and that the replacement text you type in is ended with a "\F" just like appended text.

You can also change a group of lines.

```
2,4c
```

will replace lines 2 to 4 with the lines of text you type in.

At the end of a change command, the current line is set to the last line of replacement text you typed in.

If you just type "c" without a line number, FRED assumes you want to change the current line.

Deleting Lines:

If you wish simply to delete a line rather than change it to something else, you can use the delete command "d".

```
3d
```

will delete line 3.

```
2,$d
```

will delete every line from line 2 to the end of the buffer. In other words, all that is left is line 1. If for example you use the command

```
3,$d
```

on our "Mary had a little lamb" buffer, the buffer would contain

```
Mary had a little lamb
Hickory dickory dock
```

Line 2 would now be "\$".

Just as "*p" prints the contents of the whole buffer, "*d" (or "1,\$d") deletes the contents of the whole buffer, leaving you with an empty buffer. This is a useful command if you are doing something along the following

lines. Suppose you have the first two chapters of your Great Canadian Novel stored in files called "chapter1" and "chapter2". You could sign on to FRED, read in "chapter1" and edit it for a while, appending and changing as you saw fit. When the chapter was finally perfect, you would write it back out to your file and prepare to work on "chapter2".

Now you can't read in "chapter2" right away or else it would get mixed up with the "chapter1" that's already in your buffer. Thus you say "*"d" telling FRED to delete everything of "chapter1" that's in your buffer. With an empty buffer, you can then feel free to read in "chapter2" and get on with your work.

When you delete lines in the middle of your buffer, the current line will be set to the line immediately after the last line you deleted. (You might consider the last line you deleted to be the last line you were working with, but since that line no longer exists, it can't be the current line.) When you delete the entire buffer with "*"d", the current line is set to line 0 as you will be able to see with the "=" command (if you try to print line 0 you'll get an error -- after all, there's nothing on line 0 to print). You should experiment to see what happens to the current line when you delete the last line of your buffer. Try the "=" command -- it's bound to surprise you.

It's important to remember that deletion of text makes the current line pointer (i.e. FRED's finger) point to the line AFTER the last line deleted. If you say

```
$d
a
```

to delete the last line in your buffer and to begin entering text, your text will go at the START of your buffer. Why? The reason is that "\$d" deletes the last line in your buffer and sets "." to line 0 (since there is no line after "\$"). The "a" therefore tells FRED to begin appending after line 0, at the start of your buffer. Because it's easy to become confused where your current line is after a deletion, it's always a good idea to print "." using "p" before you do anything else. Otherwise, you may get some unwanted results.

What to do When You Goof:

Everyone makes mistakes. Sooner or later, everyone types "d" when they meant to type something else and end up clobbering a line they wanted to keep. When you make this kind of mistake, do you have to retype everything you deleted by accident? No -- FRED understands human frailty and provides a way for you to undo or "void" your mistakes.

It works like this: every time FRED sees that you want him to perform a command that modifies your text (e.g. "a" to add text, "c" to change it, or "d" to delete it), FRED takes a "snapshot" of the text before he makes the modification. If you decide you want to undo the modification, type

```
v
```

(for Void) and FRED will restore things to the way they were in the last snapshot. For example,

```
a
>Little Miss Muffet
>Sat on a tuffet
>Eating her curds and whey.
>\f
d
*p
Little Miss Muffet
Sat on a tuffet
v
*p
Little Miss Muffet
Sat on a tuffet
Eating her curds and why
```

The "d" command deletes the current line, line 3. However, before FRED does the deletion, he takes a snapshot of what's in the buffer. Thus when FRED sees the V command, he backs up to that snapshot,

essentially un-doing the deletion.

The existence of the V command means that you can get out of almost any trouble you get into. If you type a command that messes things up, you can go back to the way things were just by typing V. We'll show more examples of this later.

For the moment, we'll just make a few more comments about V.

1. FRED always keeps the ten most recent snapshots. Typing one "v" backs up to the most recent; typing another "v" backs up to the snapshot before that, and so on. Thus you can back up over the effects of several commands if you want.
2. FRED only takes a snapshot when you do something that affects the content of the buffer you're looking at. Therefore FRED does not take a snapshot if you're just doing commands like "p" or "=" which only display information.
3. FRED takes his snapshots at the start of any line containing appropriate commands. If a line contains more than one command (e.g. "ddd" deletes three lines), FRED only takes a snapshot at the beginning of the line. Thus V will back up over the effects of ALL the commands on the line.
4. Sometimes you may type V to back up over some command, then decide you didn't want to back up after all. In this case, type

zv

(for Zap Void). ZV undoes the effects of the most recent V command. Thus something like

v
zv

undoes a command, then restores the effects of the command again.

5. If FRED gives you an error message, the V command undoes the effects of the commands on the line where the error occurred. Therefore whenever you get an error, you can undo any possible effects of the error by typing "v".

We hope that you will remember the V command and use it to help you out of trouble. You can afford to be a lot more adventurous with FRED and experiment with many things, once you know that you can void the effects of any command that doesn't quite work out.

Line Searches:

With the C and D commands you can now change or delete any line or group of lines in your text. One of the problems you are bound to face almost immediately is locating the lines you want to change or delete. Memorizing line numbers is annoying. Besides, line numbers change as you add or delete lines in the buffer. What you would like is a way to search through your text quickly to find the line or lines you wish to modify. FRED lets you do this by means of a "line search".

To search for a line containing the word "lamb", all you have to say is

/lamb/

FRED will reply with

Mary had a little lamb

which is the first line in the buffer containing the word "lamb". FRED makes this line the current line, so if you want to make changes all you have to say is "c". If you say "/lamb/" again, you will get

The lamb was sure to go.

which is the next line containing "lamb". All you have to do is enclose the word you want to find in slashes and FRED will look through your text for the next place that word appears.

The search for your word begins at the line AFTER the current line. Because of this, FRED will find the NEXT place where your search word occurs in the text. If FRED searches down to the end of the buffer and doesn't find your word, he will continue the search at the first line in the buffer. If FRED doesn't find the word in your text at all, he'll give you a

```
? search failed
```

error.

You can search for more than one word if you wish; in fact, you can search for any string of characters. `"/ /"` searches for two spaces. `"/you%/"` searches for the word "you" followed by a "%". We will warn you now that there are a small number of non-alphabetic non-numeric characters that have special meanings in line searches: `"^"`, `"$"`, `"."`, `"*"`, and `"["` to name a few. If you try searching for strings containing these special characters, expect unusual results (nothing bad will happen to your text, but you might not find the line you're looking for). If you try to search for the slash by typing `"/" /"` you'll have problems too -- see if you can figure out why.

Very often, you will have to search through a number of lines before you find the line you want. For example, if you are searching for "the" you must expect to see a lot of lines containing "the" before you find the line you want. To avoid typing `"/the/"` every time you wish to look further, FRED will let you type

```
//
```

When FRED sees this, he will search for the string you wanted most recently. Thus

```
/string/  
//  
//
```

will find three successive occurrences of "string" in your text.

FRED always remembers whatever you searched for last. If you have been doing a lot of appending, deleting, etc., and can't remember what you were last searching for, you can use the command

```
fe
```

(print Facts about Expressions). FRED will reply

```
/string/
```

Thus typing `"/" /"` again will start a search for the next line containing "string", even if it has been quite some time since the last search.

We might point out that you can waste a lot of time searching for a common word like "the". You'll find the line you want a lot more quickly if you search for a word or group of words that doesn't appear very often in your text. Furthermore, you should realize that FRED searches for the string of characters you want wherever they appear. For example if you're searching for "the", he will locate lines containing words like "they", "them", "other", and so on: anywhere the letters "t", "h", and "e" appear in that order.

If you want to search for the first occurrence of a string BEFORE the current line, you can type

```
-/string/
```

When FRED sees this, he will begin searching backwards through the buffer for "string". If he gets to the first line in the buffer without finding "string" yet, he will continue searching backwards from the end of the buffer. Again if he can't find "string" at all, he will give you an error message.

In all line searches, both forwards and backwards through your text, FRED will ignore whether letters are in upper or lower case. Thus the line search

```
/this/
```

will find lines containing "this", "THIS", "This", "tHIs", and so on.

Exercise 4:

Read or append some lines into your buffer. Use a line search to search through the buffer for a particular line. Change that line using the C command. Experiment with changing and deleting lines. Don't forget that you can use "*p" to show what has happened to the buffer after a number of changes and deletions.

The S Command:

One of the most common editing jobs is the correction of typing errors. The C command can be used to correct such errors as we have already seen. However, typing in an entire line is a lot of work to correct one small error (and if you aren't careful, you may make more mistakes the second time through). What you would like to do is tell FRED to change a single word or group of words without having to change an entire line.

This is done with the S command (S stands for substitute). Suppose we have typed in

```
Mary had a littel lamb
```

We would like to be able to change the "littel" to "little". You do this with the command

```
s/littel/little/
```

This tells FRED to substitute "littel" with "little" everywhere in the current line. After you've done the substitution, you can print the line with "p" and see that it now reads

```
Mary had a little lamb
```

as it's supposed to.

We didn't really have to type as much as we did in the S command above. We could have written

```
s/el/le/
```

and it would have changed "littel" to "little". In fact, we can change any string of characters to any other string.

Suppose you try the substitution

```
s/a/the/
```

on "Mary had a little lamb". You might think that you'd get "Mary had the little lamb", but you'd be wrong. If you print the line using "p", you'll see you have

```
Mthery hthed the little lthemb
```

What has happened is that FRED has changed every "a" in the line to "the". If you want to change just the word "a" to "the", you might type

```
s/ a / the /
```

This will change an "a" surrounded by spaces into "the" surrounded by spaces. If you left out the spaces around "the" you would get "Mary hadthelittle lamb". Mistakes like this are very easy to make, even for experienced FRED users. It's good to get into the habit of printing a line after you've made a substitution in it, just to check that you haven't made a mistake.

One mistake that beginners sometimes make is thinking that there is a space at the end of a line. You can certainly type in space characters at the end of your lines if you want, but the spaces aren't put in automatically. If you try the statement

```
s/lamb /duck /
```

on "Mary had a little lamb", you'll find you'll get an error. The reason is that there is no space character after "lamb", even though there may be a lot of space after "lamb" when the line is printed on the terminal. In your text, there is actually a "new-line" character after the word "lamb". This "new-line" is an invisible character which tells FRED that one line is finished and a new line is about to begin. Every time you type in a carriage return, a "new-line" character is automatically put into your text to keep lines separate.

We have already said that the S command without a line number makes its changes in the current line. If you provide a line number as in

```
2s/dog/cat/
```

the substitutions will be made only on the line you specify. In the above case, every "dog" in line 2 would be changed to "cat". You can also make substitutions on a range of lines, as in

```
3,5s/boo/hoo/
```

which changes "boo" to "hoo" in lines 3 to 5 inclusive. This is very handy if you decide to change the name of the heroine of your Great Canadian Novel while you're editing. The command

```
*s/Alice/Ethel/
```

will change the name everywhere it appears in your text. (In place of "*" you could also have used "1,\$".)

If you give FRED an S command to change one string into another string and FRED can't find the first string in the lines you specify, you'll get an error message

```
? no text changed
```

Most of the time this happens because you've typed in the first string incorrectly.

By the way, it doesn't matter whether you type the first string of an S command in upper or lower case.

```
s/this/x/
```

will change "this", "THIS", "tHIs", and so on. The second string (i.e. the one that's being put into your text in place of the first) should be in the case you want it to appear in the text. Thus, for example,

```
s/h/h/
```

changes every "h" (upper OR lower case) into a lower case "h". The first string is really just a *pattern* indicating the approximate "look" of all the strings you want to change.

You should spend a considerable length of time experimenting with the S command. It is a very convenient command and easy to use once you have a little experience with it.

More Complicated Substitutions:

The substitutions we have been speaking about so far are very simple. One string is replaced with another string everywhere the first string occurs on the line. We are now ready for more ambitious substitutions.

A simple variation of the ordinary S command has the form

```
s3/a/the/
```

The "3" after the "s" tells FRED to change only the third "a" in a line into "the". This command will change "Mary had a little lamb" into "Mary had the little lamb", since only the third "a" is changed. We could also have said

```
s-2/a/the/
```

This command changes only the second "a" from the end of the line. Thus you see that a positive number <n> after the "s" changes only the <n>th occurrence of the first string. A negative <-n> changes the <n>th occurrence of the first string from the end of the line.

We are now ready to speak of certain special characters that can be used in the S command. These characters are called "pattern" characters in FRED and the first string in an S command is called the pattern that is changed.

The first special character we will look at is "^". "^" matches the beginning of a line. Thus if we said

```
s/^/a/
```

we would be putting an "a" at the beginning of the line. This is NOT the same as changing the first character in the line to "a". "^" is actually an invisible character that appears before the first character in the line. Thus

```
s/^/My friend /
```

would change "Mary had a little lamb" to "My friend Mary had a little lamb".

You can also use "^" with more characters after it. For example,

```
*s/^%/?/
```

would change all "%" signs at the beginning of text lines into "?" marks. Note that the "^" doesn't appear in the string which replaces "%".

Just as "^" represents an invisible character at the beginning of a line, "\$" represents an invisible character at the end of a line. Thus

```
s/$/ and so on./
```

would change "Mary had a little lamb" to "Mary had a little lamb and so on."

```
*s/:$/;/
```

would change any ":" at the end of a line into a ";".

The next special pattern character we'll look at is ".". When "." is used in the pattern side of an S command, it stands for ANY character. For example, try

```
s/./x/
```

on the line "Mary had a little lamb". The result will be

```
xxxxxxxxxxxxxxxxxxxxxxxx
```

which is not all that inspiring a line. Notice that EVERY character, whether it's a letter or a space, is turned into an "x". Obviously, you have to be careful when you use the "." in a pattern. Many beginners have come to grief trying to change a period to a comma with the instruction "s/./," and ending up with a line of commas.

However, the "." can be a great saver of typing when used carefully. Suppose you want to delete the last character of every line in your text. All you have to tell FRED is

```
*s/.$//
```

Literally, this means to find any character immediately followed by the end of the line and replace it with nothing. This is a great deal easier than going through your text line by line to find the exact character that ends the line and then deleting that character specifically.

Another time you can use the "." is when your text contains a line like

I have \$5 in my pocket.

and you'd like to change "\$5" to the words "five dollars". If you say

```
s/$5/five dollars/
```

FRED thinks of the special meaning of "\$" and believes you want him to look for the end of the line followed by the character "5". Obviously, this will give you an error. However, the command

```
s/.5/five dollars/
```

tells FRED to look for any character followed by a "5". This matches your "\$5" and your text is changed in the way you want.

The last special pattern character we'll take for now is "*". Try the command

```
s/./hello/
```

on any line and you will find that the line has been changed simply to

```
hello
```

Obviously "." tells FRED to change everything!

The reason is that any character followed by "*" matches any string of those characters. For example, "b*" matches "b", "bb", "bbb", and so on. It also matches "" (nothing) which counts as zero "b"s.

Let's take a simple example where the "*" can come in handy. Very often, text lines will have blanks on the end which take up space inside the computer but really don't have any purpose. If you could delete those blanks, you could save all that space in your files. The command

```
*s/ *$//
```

will go through all your text and delete any number of spaces FRED finds at the end of each line. There will be no problem with lines that have no spaces on the end, since " *" matches zero spaces too.

Getting back to our first example, think what "." will match. "." matches any character, so "." will match any number of characters (including no characters at all). If you use the command

```
s/y./s/
```

on the line "Mary had a little lamb", what will happen? FRED will find a "y" and any number of characters after the "y" and replace the whole thing with just "s". Thus the command above turns "Mary had a little lamb" into "Mars".

The "." construction can be used to save typing in many different ways. The command

```
s/y.*la/y possessed a tiny la/
```

changes "Mary had a little lamb" into "Mary possessed a tiny lamb". Thus when you are changing a large portion of a line, you need only type the first little bit and the last little bit, and use "." to stand for what's in between.

Given the choice, a "*" pattern will match the longest string possible. For example,

```
s/a.*a/x/
```

will change "Mary had a little lamb" to

```
Mxmb
```

"a.*a" was taken as everything from the first "a" to the last "a" in the line, the longest string in the line beginning and ending with "a". Generally speaking, this works is exactly what you want -- you just have to be careful from time to time.

Exercise 5:

Experiment with the S command. Use all the different pattern characters, "^", "\$", "." and "*". Try these pattern characters in line searches too. For example,

```
/^begin/
```

tells FRED to search for a line beginning with "begin". Make sure you spend a considerable length of time learning how all these characters work. The S command is a command you will be using all the time in editing with FRED, and knowing how to use the special pattern characters can save a good deal of time.

Escaping:

We come now to the idea of "escaping" characters. Those with some experience of other computer languages may already understand this concept, but for beginners we will explain the notion in some detail.

There are just so many characters on the terminal keyboard, and most of them are used by FRED for something. Almost all the letters are FRED commands. The numbers and arithmetic symbols like "+" and "-" are used for arithmetical operations. We have already seen that "^", "\$", "." and "*" have special uses in FRED, both as pattern characters and line number shorthands. Many of the other characters on the keyboard have special uses we haven't spoken of yet.

Eventually the designers of a language come to the point where they want to install more special features than there are characters on the keyboard. In FRED this problem was solved by defining a new set of symbols which are typed in as two characters. The first character is called the "escape" character (this is NOT the same thing as the key marked "escape" on some terminals). In FRED, the escape character is the backslash "\". When FRED sees a backslash followed by another character, he takes the two as ONE symbol. Inside the computer the combination is stored as a single character, even though you type it in as two. This is all done simply because there aren't enough characters on the keyboard to do everything we want, so we have to manufacture a few characters ourselves.

You have already seen one example of this sort of symbol. "\F" is the symbol you use to end text input after an A or C command. Another symbol you are apt to use frequently is "\C". (Remember, this is just ONE character as far as FRED's concerned, even though you type it as two.) "\C" tells FRED that the next character is to be taken literally, without any special meaning. A character preceded by a "\C" is said to be "escaped".

Now we'll give a few examples of how escaping works. Normally

```
s/^a/
```

would place an "a" at the beginning of a line. If you say

```
s/\C^a/
```

you are telling FRED to ignore the special meaning of the character "^" (the beginning of the line) and take "^" literally. Thus the second S command would change the character "^" into an "a". In the same way,

```
s/\C$/x/
```

changes the character "\$" into an "x", and ignores the special meaning "\$" usually has in S commands. To change a period into a comma, you could say

```
s/\C./,/
```

"\C" can be used for this same purpose in line searches; for example,

/\C*/

searches for a line containing a "*".

One other use of "\C" is in breaking up long lines into two shorter lines. By typing

```
s/ab/a\C
b/
```

you can put a "new-line" character in between the characters "a" and "b", thus breaking the line in two at that point. Normally, the <cr> is the character which tells FRED there's work to do; however, the "\C" in front of the <cr> tells FRED to treat it as just another character. If you tried to type

```
s/ab/a
b/
```

without the "\C", FRED would take a look at the first line, would decide you had left something out, and would give you an error before he even saw the "b/" on the next line.

We will talk about more of these special symbols which use "\" as they become useful to us. For now, we will just remind you that to enter the character "\" into your text, you have to type two, as in "\". If you only type one, FRED thinks you are talking about one of the special symbols which are made up of "\" plus another character, and surprising things could happen.

Some More Useful Commands:

We will now describe a few more commands that are also useful for editing with FRED.

The I command (Insert) is similar to the A command, except that it places your text before a line, not after it. For example, if your buffer contains

```
Hey diddle diddle
The cat and the fiddle
The cow jumped over the moon
```

you could type

```
2i
>Sing a song of sixpence
>A pocketful of rye
>\F
```

to get the following:

```
Hey diddle diddle
Sing a song of sixpence
A pocketful of rye
The cat and the fiddle
The cow jumped over the moon
```

Note that a "\F" is required to end your text input just as with the A and C commands.

The G command will execute a list of other commands for a selection of lines in your buffer. You select the lines with the same kind of pattern you use in S commands and line searches. For example,

```
g/^%/p
```

goes through your text looking for lines beginning with "%" and prints them. The command

```
g/end$/ s/end/finish/p
```

goes through your text looking for lines which end in "end". On these lines it performs the S command changing "end" to "finish" and then it performs the P command printing the line. A command like

```
*s/end$/finish/
```

would make the same substitutions as the Global command, but it wouldn't print out the lines in which it made substitutions.

You can put any instructions you want in a Global command, including line searches. For example

```
g/^hi/p .+1p .+1p /hello/p
```

goes through your text looking for lines beginning with "hi". When it finds one, the "p" prints the line, the first ".+1p" prints the next line, and the next ".+1p" prints the line after that. Lastly, the "/hello/p" tells FRED to search through the buffer and print the next line that contains "hello". (Note that this command would give you an error if either of the last two lines in the buffer started with "hi", since the ".+1p" commands would try to print lines beyond the end of the buffer.)

There is another form of the Global command which will perform your list of commands for all lines which DON'T match a given pattern. In this form, you put a "~" after the "g" as in

```
g~/?/p .+1d
```

This command looks for any line which does NOT contain a "?", prints the line, and deletes the line after. (Again, you might get an error with this if you go past the end of your buffer.)

If you don't specify any line numbers for a G or G~ command, FRED will scan your entire buffer for lines which match or don't match the pattern you give. You can also specify a range of lines, as in

```
2,5g/^/ s/^/;/ s/$/:/
```

This command (which might look a little like gibberish at first) tells FRED to look through lines 2 to 5 inclusive for a beginning of line character "^". Obviously, all lines begin this way, so the commands following the "g/^/" will be done for every line from 2 to 5. The first S command puts a ";" at the beginning of the line and the second puts a ":" at the end. Thus the given command will make sure that lines 2 to 5 begin with ";" and end with ":". You can also do a Global command on just one line, though we're not sure why you'd want to.

The last command we will speak about in this section is the "!" command. When FRED sees an "!" at the beginning of a command, he sends the rest of the line to the TSS system for processing. For example,

```
!list /myfile
```

will list the contents of "myfile" on your terminal, just as if you had replied "list /myfile" to the system's "*" prompt. When the listing is finished, another "!" is printed and you will be back in FRED again. This is convenient, for example, if you are preparing a report using the TF text formatter. You can say

```
!tf myfile >report
!
!list report
...report listing...
```

This formats "myfile" into another file called "report" and then lists "report" on your terminal. If there is something wrong with the listed report, you can edit "myfile" using FRED, and then TF it again. In this way, you can use all the facilities of TSS without ever leaving FRED.

Exercise 6:

Read in a file using the R command. Insert some text with the I command, and make sure that the text contains special pattern characters such as "^", "\$" and "*". Do some line searches for these characters using the "\C" symbol to escape their special meanings. Do some Global operations, such as putting a "%" at the start of every line and a "?" at the end. Make sure you use the G command on subsections of your text as well as the whole buffer.

Some Handy Short-forms:

In this section, we will learn about some features of FRED which help cut down on your typing. We'll first look at line addresses.

We have already spoken about line numbers and the special characters "." (the current line), "\$" (the last line in the buffer), and "*" (which is short for "1,\$", every line in the buffer). Line numbers and these special characters are forms of Line Addresses. A line address is just a way of referring to a particular line of text to tell FRED where he's supposed to do something.

A line search like `/string/` is another form of line address. As we have said before, typing

```
/string/
```

will cause FRED to find the next line containing "string" and print it. We could also say

```
/string/s/string/rope/
```

This tells FRED to find the next line containing "string" and then change that "string" to "rope".

```
/^hello/d
```

will delete the next line beginning with "hello". You can also use an instruction like

```
-/end$/a
```

This command tells FRED to search backwards through the buffer to find the first line before "." ending in "end", and then to begin appending text after that line.

Thus we see that line searches like `/string/` and `-/rope/` can be used in the same way as line numbers and the special line address characters we know. We can even say something like

```
-/hello/,/goodbye/p
```

This tells FRED to search back through the buffer for a line containing "hello", to search forward from the "hello" line for a line containing "goodbye", and finally to print everything from the "hello" line to the "goodbye" line.

A different kind of line address is the symbol "^". This is shorthand for "-1". Thus the command

```
^
```

will go up a line (to the line before the current line) and print it. The command

```
^^
```

will go up two lines from the current line, and so on. Thus "^" lets you go backwards through a buffer as easily as `<cr>` (meaning ".+1") lets you go forwards.

The next special line address character we will look at is "&". "&" is a convenient shorthand for the pair of addresses ".+1, +20". Just as "3,5" is short for "3,5p", "&" is short for "&p" or ".+1, +20p". In other words, just typing

```
&
```

will print the twenty lines immediately following the current line. If there aren't twenty lines before the end of the buffer, the command "&" will print from ".+1" down to the end of the buffer and then stop.

Using "&" is a very convenient way of reading through your text. For example,

```
1  
&
```

will print the first line of your text and then lines 2 to 21. To print the next twenty lines, just type "&" again. In this way, you can look through a file in groups of twenty lines at a time. This is often more desirable than going through one line at a time with <cr> (which is slow) or going through the file all at once with "*p" (which makes for difficulties if you happen to be working at a CRT terminal that prints faster than you can read).

Another form of line address is typed as "-&". This is equivalent to "-20". If, for example, you type

-&&

the "-&" will tell FRED to back up twenty lines and the second "&" will tell FRED to print twenty lines. Thus "-&&" prints twenty lines of text ending with the current line. This is very handy if you want to read what you've just been typing in. You can also say

\$-&&

This command prints the last twenty lines in your buffer, a good way to see where you left off typing your text if you can't remember.

Substitution Short Forms:

We have mentioned that a command like

s/h/h/

turns every "h" in a line into a lower case "h", whether the original "h" was upper or lower case. In the same way,

s/h/H/

puts every "h" into upper case.

A shorter way of doing the same thing is with the ZU (Zap Upper Case) and ZL (Zap Lower Case) commands.

3,5zu/1/

will put every "l" in lines 3 to 5 inclusive into upper case.

\$z1/j/

will put every "j" in the last line into lower case. You can zap the case of any string of characters. For example,

*zu/fred/

will turn every "fred" in your buffer into "FRED". It also capitalizes "Fred", "fREd", and so on.

If you don't specify any string after a ZL or ZU command, the entire line is put into one case or the other.

*z1

17,\$zu

first puts every line in the buffer into lower case, and then puts lines 17 to the end into upper case.

Just as the S command gives an error if it cannot make any substitutions, ZL and ZU give an error if they do not change the case of any letter.

We should point out that ZL and ZU change only letters. "zu/6/" does not give you the character which you'd get on your terminal by typing a "6" with the shift key. ZL and ZU simply leave non-alphabetic characters

alone. For this reason, "zu/6/" will always get an error because it will never change anything. the command doesn't make any changes to anything.

We should point out that ZL and ZU are really disguised versions of the S command. Thus you can use all the special pattern characters we have spoken about in ZL and ZU commands, just as you can use them in S commands. Thus

```
*zu/^./
```

capitalizes the first letter of every line in the buffer, and

```
^z1/a.*$/
```

puts all letters in the line before the current line into lower case, beginning at the first "a" in the line and continuing to the end of the line.

Another short form which can be used in connection with substitutions is the null pattern "/". We have already mentioned that in line searches "/" refers to the last string you were searching for. In fact, "/" refers to the last pattern FRED has seen, whether in a line search, an S command, a ZL command, or a ZU command. Thus

```
/hello/  
s//hi/
```

searches for a line containing "hello" and then changes "hello" to "hi". The "/" in the pattern part of the S command stands for the last pattern FRED has seen, and that pattern is "hello". In the same way

```
zu/fred/  
//
```

turns "fred" in the current line into "FRED" and then searches for the next time "fred" appears in your text.

Remember, "/" refers to the last pattern FRED has seen, whether in an S, ZL, ZU, G, or line search. If you are not sure what that pattern is, you can use the FE command to have FRED remind you.

Another useful short form in connection with the S command is "&". When an "&" appears on the right hand side of an S command, it stands for the string that was matched by the left hand side. For example,

```
s/abc/&d/
```

will change "abc" to "abcd" since the "&" stands for "abc" in this instance. The command

```
s/xyz/&&/
```

will change "xyz" to "xyzxyz" because of the two "&" characters.

Because "&" has a special meaning on the right hand side of an S command, you have to use a "\C" to refer to the "&" character itself. For example,

```
s/ampersand/\C&/
```

changes the word "ampersand" into the character "&". Just typing

```
s/ampersand/&/
```

won't do anything; the word "ampersand" will be replaced by itself!

We have mentioned before that it is a good idea to print the results of every substitution you make, just to make sure that the substitution did what you wanted. You can save some time by typing the S and P commands on the same line as in

```
s/this/that/p
```

The S command does the substitution and the P command that follows prints the current line. Since the current line was the one where you just made the substitution, the "p" prints the results of the substitution. If you try a command like

```
*s/this/that/p
```

the only line printed will be the last line in which "this" was changed to "that". The P will only print whatever line is current after the S is finished. If you want to print every line where a substitution is made, you have to use a command like

```
g/this/s/this/that/p
```

This is equivalent to

```
g/this/s//that/p
```

Short Forms for Reading and Writing:

FRED remembers the name of the last file you used in either a read or write operation. To see the name of this file, use the command

```
f
```

which stands for "Facts". Make sure there is no space between the "f" and the <cr> immediately after. FRED will say something like

```
b(0) rvanwinkle/myfile
```

where "rvanwinkle" is your userid and "myfile" is the name of your file. We'll explain what the "b(0)" means in a later section.

It's also possible that there will be a "?" printed after the file name. This means that you have changed your buffer since the last time you read a file in or wrote your buffer out. Essentially then, the "?" means that there have been changes made to your text that haven't been saved in a file. This is just a reminder that you should write out your text to preserve the changes you have made.

If you want to write your text to the file name associated with your buffer, you don't have to type that name out in the W command. All you have to say is

```
w
```

(make sure there is no space between the "w" and the <cr> which comes after). If you want to read from the file associated with your buffer, all you have to say is

```
r
```

If you want to read or write a different file than the one associated with your buffer, all you have to do is specify the proper file name in your R or W command as we have described before. If there are no line addresses specified for the R or W command, the file name associated with your buffer will be changed to this new name.

You can also change the file name associated with your buffer by saying

```
f /newfile
```

This will change the associated file name to "/newfile". The next time you do a read or write using just "r" or "w", FRED will use "rvanwinkle/newfile" for the operation.

Exercise 7:

Read in a file and use the F command to check that the name of the file is associated with your buffer. Make some substitutions in the text, using the "&" character and the null pattern "/". Also try the ZL and ZU commands. Print your file on the terminal using the "&" line address character. Use some line search patterns as line addresses for S commands.

Multiple Buffers:

Up until now we have always been working with a single buffer. FRED allows you to have as many buffers as you want. Each new buffer is exactly like the one buffer you have been using all along. Every buffer can contain a huge number of lines. Every buffer has one line which is the current line ".". Every buffer can have a file name associated with it for reading and writing.

To create a new buffer, all you have to do is think of a name (say, "mybuff") and type the command

```
b(mybuff)
```

(where B naturally stands for Buffer). Notice that the name of the new buffer is enclosed in parentheses. A buffer name can be up to sixteen characters long, but wherever the name is used, it must be contained in parentheses. Usually we think of the parentheses as part of the buffer's name.

Suppose now that you have given FRED the B command above. What happens if you try "*p"? You'll find that you'll get an error message. Why? Because you have entered a new buffer and there's nothing in this buffer to print yet! You might argue that you had plenty of text in your old buffer, and this may be perfectly true. Issuing a B command is like telling FRED to set aside one buffer and to look at another, just as you might set aside one piece of paper when you want to work on another. You can only work on one piece of paper at a time, and FRED can only work with one buffer at a time. Like a piece of paper which has been set aside, the old buffer is still around somewhere; you just don't have it in front of you at the moment. As far as FRED is concerned, there is only one buffer out of the many buffers you might have created which is "the current buffer". You can change current buffers by using the B command, but you can't have more than one buffer in front of you at a time.

When you enter FRED, you are automatically given a buffer named "(0)". Remember the "b(0)" that was printed out by the F command along with the name of the file associated with the buffer? That was just telling what the name of your current buffer was. If you type in

```
b(mybuff)
f
```

FRED will reply

```
b(mybuff)
```

He won't print out a file name because this new buffer doesn't have a file name associated with it yet.

You can use the new buffer exactly the same way you used the old one. You can read in files, write out your text, append, change, delete, make substitutions, and so on. Those commands will only affect your new buffer; as we said, your old buffer is out of the picture for the moment. Of course, you can always go back to the old buffer just by saying

```
b(0)
```

This command will put you back into buffer "(0)" in exactly the same place you were when you left -- your current line will be exactly the same as it was before you issued the "b(mybuff)" command. Furthermore, if you go back to "(mybuff)" you'll find that (mybuff)'s current line is the same as it was when you last left the buffer. FRED remembers where his finger was pointing in a buffer whenever you return to a buffer.

You might ask why you would need to have more than one buffer when you can do so much editing with a single buffer. One application might be if you're preparing some text which can be broken up into several parts, e.g. a document with several sections to it, a computer program with several subroutines, or data from several different sources. If you keep each part in a separate buffer you can edit them all separately (which

tends to be faster since FRED is working with smaller files) and yet you still have all the parts available if you want to refer from one to the other.

If you are working in one buffer and wish to print another, you can use the ZP (Zap Print) command. For example,

```
zp(part5)
```

will print the contents of buffer "(part5)" regardless of which buffer you have current. Of course you can also switch to buffer "(part5)" using the B command, look at whatever you want to, and return to the buffer where you were working.

Transferring Text from One Buffer to Another:

While most of FRED's commands only operate on the current buffer, some allow you to move text from one buffer to another. The first of these is the M (Move) Command. Typing

```
3,5m(lines)
```

moves lines 3 to 5 inclusive out of the current buffer (whatever it is) into a buffer called "(lines)". If "(lines)" has not been created before, this command will create the buffer. If "(lines)" was around before you issued the Move command, any text that was in "(lines)" previously is wiped out as the new text is moved in. Thus you should be careful that you don't move lines into a buffer that already contains text you want to keep. You can also move a single line, as in

```
$m(end)
```

which moves the last line of the current buffer into a buffer called "(end)". Anything that was previously in "(end)" is wiped out in the process.

The Move command is handy if you have a large amount of text that you want to break up into sections. Suppose your sections all end with the words "end of section". You could type a series of commands like

```
1,/end of section/m(section1)
1,/end of section/m(section2)
1,/end of section/m(section3)
```

and so on to break up your text. Notice that once Section 1 has been moved out, line 1 of your current buffer will have the first line of Section 2. When Section 2 is gone, line 1 will be the first line of Section 3. Thus the next section to move will always start at line 1 and end at the words "end of section".

The K (Kopy) command is similar to the M command, except that it copies lines of text from the current buffer instead of moving them. Thus

```
^,.+3k(hi mom)
```

copies lines ".-1" to ".+3" into a buffer called "(hi mom)". Anything that was in "(hi mom)" previously is wiped out as the lines are copied in. You can also copy a single line as in

```
-&k(frodo)
```

where line ".-20" is copied into a buffer called "(frodo)".

The M and K commands wipe out the contents of any buffer where lines are moved or copied. Often you would like to be able to move lines from one buffer into another while keeping the contents of the second buffer intact. One command that does this is the ZA (Zap Append) command.

```
$za(stuff)
```

appends the contents of buffer "(stuff)" immediately after the last line of the current buffer. The text in "(stuff)" is not changed at all. Just as we used the M command to break a report up into sections a little while

ago, we can reassemble the report using Zap Append. For example,

```
b(fullreport)
$za(section1)
$za(section2)
$za(section3)
```

and so on creates an assembled version of the report in a buffer called "(fullreport)". (In case you're interested, this is how the sections of this tutorial were assembled into the final product.)

The FB Command:

By now, you might be wondering how you're going to keep track of all the buffers you've created while editing. The answer is simple: FRED keeps track for you! Just type

```
fb
```

(for "Facts about Buffers") and FRED will list the following information: the name of each buffer that has anything in it, beginning with the current buffer; the value of "." in each buffer; the value of "\$" in each buffer; the file name associated with each buffer; and whether a buffer is questionable or not (i.e. whether the buffer has been modified since the last read or write). For example, you might have

```
fb
b(henry) 23,45 rvanwinkle/snore1
b(hudson) 66,66 rvanwinkle/snore2 ?
```

This says that your current buffer is named "(henry)", that your current line is 23, that there are 45 lines in this buffer altogether, and that the file name associated with this buffer is "rvanwinkle/snore1". You also have a second buffer named "(hudson)". Its current line is the last line of the buffer, line 66. The file name associated with buffer "(hudson)" is "rvanwinkle/snore2" and the "?" means you have changed the contents of "(hudson)" since the last time you read or wrote "snore2".

ZM and ZK:

The last commands we will discuss in this section are ZM (Zap Move) and ZK (Zap Kopy). ZM moves lines from the current buffer and appends them after "." in another buffer. For example,

```
4,$zm(lunch)
```

moves all the lines of the current buffer from line 4 to the end into a buffer called "(lunch)". Thus the current buffer will be left with only three lines. As for "(lunch)", the lines that are moved in are appended after the current line, whatever that line is. The FB command is a good way to find out what the current line in "(lunch)" is. Of course,

```
b(lunch)
p
```

will also tell you what the current line is, by going into lunch and printing out whatever line is current.

The ZK command is similar to ZM except that the given lines are copied from the current buffer, not moved right out.

```
&zk(albatross)
```

copies the next twenty lines from the current buffer into a buffer named "(albatross)", appending the copied lines immediately after the current line in "(albatross)".

Quitting with Multiple Buffers:

You'll remember that FRED wouldn't let you quit with "q" when you had a single buffer which hadn't been written out to a file since it was last changed. If you have more than one buffer, FRED won't allow you to quit with "q" if ANY of your buffers have been changed since they were last read or written. Again, the reason for this is simple: FRED assumes you wouldn't be making changes to your text if you didn't want to preserve those changes on file.

However, things aren't always easy to keep track of with multiple buffers. You may make changes in one buffer, transfer that text to another buffer using K, M, ZA, ZM, or ZK, and then write out the second buffer. Your changes HAVE been preserved but FRED doesn't realize it.

If you attempt to leave with "q" and FRED won't let you, use the FB command to see if you have any buffers containing text that hasn't been preserved on file yet. If you are certain that you have saved all that you want to save, then the command

qq

will tell FRED to quit even if some of your buffers are marked with a "?" in your FB listing.

We would recommend that you avoid using "qq" at all, and certainly never use it before you have used the FB command to make sure that you have saved everything you need. If you get in the habit of using "qq", sooner or later you're bound to quit before you've saved something important and that means you'll have to do a lot of work all over again. Not even the V command can help you if you make this kind of mistake.

Exercise 8:

Create a number of buffers with names of your own choosing and put some text in them using R and A. Use FB to list the buffers you have created. Transfer text from one buffer to another using M, K, ZA, ZM, and ZK. Don't forget that you can print the contents of any buffer with the ZP command. After every text transfer, check to see what has happened to "." in your current buffer.

Options:

The user has a certain choice of options to control the way FRED operates. Up until now, we have assumed that you have been using the standard (default) options for FRED. In this section, we'll describe a few of the ways in which you can change the way FRED works.

Paging:

For large printing jobs like "*p" and "fb" (if you have a lot of buffers), it is often convenient to be able to have FRED print a small number of lines, then wait until you indicate that you've read what's been printed and are ready to see more.

The way to have FRED do this is to type something like

o+p20

The "o" stands for Option, the "+p" says turn on paging, and the "20" means you want pages which are twenty lines long (this is about the size of many CRT screens). Of course you can specify any number in place of the "20".

Now when you do a "*p", FRED will print out twenty lines and pause. To get the next twenty lines, just type a <cr>. In this way, you can read through the text in your current buffer twenty lines at a time. If you want to stop this printing before you get to the end of your text, just type anything other than a <cr>. FRED will beep, type out a "?", and then take any new command you give him. When FRED is pausing in the middle of printing like this, the cursor of your terminal will NOT advance one line beyond the last line printed; when FRED is finished printing, it will advance. This is the way to tell whether FRED is finished or just waiting for you to tell him to continue.

Using "o+p" to set this "pagesize" also affects the use of "&" as a line address. When the pagesize is set, "&" has the meaning ".+1,.,+pagesize". Thus if you set yourself a pagesize of ten lines with

```
o+p10
```

"&" would be equivalent to ".+1,.,+10".

If you wish to change the pagesize at any time, all you have to do is type in "o+p" followed by the pagesize you want. If for some reason you want to turn off this paging feature, just type

```
o-p
```

and FRED will return to his previous method of printing.

String Options:

We have said previously that FRED does not care whether letters are in upper or lower case in line searches and on the left hand side of S commands. If you want FRED to pay attention to the case of letters in these situations, type

```
o-sd
```

(where the "-" stands for "turn off" and the "sd" for "string duality"). From this point on, FRED will pay attention to the case of letters: "/this/" will only match "this" and not "THIS", "tHIs", and so on. The command

```
o+sd
```

turns on "string duality" again and FRED will once more ignore the case of letters.

We have already mentioned that you can tell FRED to ignore the special meanings of pattern characters like "^" and "\$" by using "\C". However, "\C" only works once, for the pattern character immediately following it. If you want to deactivate a pattern character's special meaning permanently, you can say (for example)

```
o-s^
```

This tells FRED to forget about the special meaning of "^" in S commands and line searches. With this option, a line search like "/^/" will search for the character "^", not the beginning of a line. You can turn off as many of these special pattern characters as you want. For example,

```
o-s$.*
```

turns off the special pattern meanings of "\$", "." and "*". To turn on these special meanings again, type

```
o+s^$.*
```

or any combination of the special characters you want.

The last special character we spoke of in connection with the S command was "&", standing for the string that was matched by the S command. If you want to turn off this special meaning,

```
o-s&
```

will do the trick. Once you have issued this command,

```
s/xyz/&&/
```

will turn "xyz" into "&&", not "xyzxyz" as before. To turn the special meaning of "&" back on, use

```
o+s&
```

Input Options:

If you don't like the ">" that FRED uses to prompt for text input after A, C, or I commands, you can turn off this prompting with the command

```
o-ia
```

This means don't prompt for Input when Appending. If you'd prefer FRED to say something else when prompting for text (e.g "text, please->"), you can tell him to do this with the command

```
o+ia/text, please->/
```

From now on, FRED will type

```
text, please->
```

when he's ready for another line of text input. Touch-typists often find it convenient to use the <bell> (control g) as a prompt character -- this way, FRED will beep when he's ready to take more text and you don't have to keep your eyes on the screen for a visual prompt. If you have turned off prompting, FRED will just give a line-feed when he's ready for more text input.

If you don't like "\" as your escape character (some terminals don't even have a "\"), you can change your escape character using the "O+IE" command. For example,

```
o+ie'
```

sets your escape character to '. Whatever your escape character, remember that you have to type TWO of them to represent the character itself. When FRED sees a single escape character, he assumes you're constructing one of those special symbols that consist of the escape character followed by another letter. If you change your escape character to ', remember too that you have to type "'F", "'C", etc. from then on.

Tabs:

If your terminal doesn't have a tab key, you can still put tabs into your text by typing "<control>i" where <control> is the "control" key of your terminal (hold down the CTRL key and then type "i"). FRED will also let you type in a tab as "\", the backslash followed by a space. The computer stores each tab as a <tab> character, the same as any other character like "a" or "!". However, FRED is clever enough to expand a <tab> character into the correct number of spaces when he types lines out to your terminal, provided that you tell him what the correct number of spaces is.

When you first enter FRED, tab stops are set every four columns along the line, i.e. in columns 5, 9, 13, 17, etc. You can change this arrangement using the "O+T" command.

The simplest form of this command is just "O+T<n>" where <n> is a number. This command sets tab stops every <n> columns along a line, beginning at <n>+1. For example,

```
o+t6
```

sets tab stops in columns 7, 13, 19, 25, etc.

A second way to set tab stops is to specify the numbers explicitly. For example,

```
o+t5,10,15,20,24
```

sets stops in columns 5, 10, 15, 20, and 24 (naturally!). In addition, however, FRED takes the difference between the last two numbers and sets tab stops all along the rest of the line at intervals of that difference. Since the difference between 20 and 24 is 4, the command above would also set tab stops in columns 28, 32, 36, 40, etc. If you want tab stops ONLY in the lines you specify, you have to type the last number twice (so that the difference between the last two numbers is zero). Thus

o+t5,10,10

sets tabs in ONLY columns 5 and 10, while

o+t5,10

sets tabs in columns 5, 10, 15, 20, etc.

Tab stops are local to each buffer. This means that different buffers can have different tab stops. When you create a buffer using B, K, or M, the new buffer inherits its tab stops from the current buffer. Once the buffer is created however, you can use the "O+T" command to set new tabs.

Remember, the tab stops set by "O+T" only tell FRED how to format a line when he is printing something out to your terminal. Inside the computer, the tabs are stored as <tab> characters, NOT the correct number of spaces. If you want to turn tabs into spaces, the command

zo

will Zap Out your tab characters and convert them into the proper number of space characters. On the other hand, if you want to turn spaces into tabs you can use the ZI (Zap In) command by just typing

zi

To tell FRED not to expand tabs into spaces when printing to the terminal, you can use the instruction

o-t

Once FRED sees this, he will begin printing tabs as <tab> characters rather than the proper number of spaces. What happens in this case depends on your terminal. Some terminals print <tab> characters as single spaces, some terminals have pre-set tab stops, and some terminals do not print tabs at all.

The FO Command:

Typing

fo

(Facts about Options) will provide a listing of the options which are currently in effect in FRED. We have not described all the available options by any means, and therefore some of the lines which FO prints may not mean much to you. The complete set of FRED options available are described in the FRED Reference Manual.

Conclusion:

If you have read through this tutorial, done the exercises, and experimented with FRED on your own, you are very well-equipped to do just about any editing job you want. Now's the time when we tell you that you've barely scratched the surface of FRED's capabilities. The Appendices of this tutorial will tell you a little bit more about the language; however, to find out about FRED's full potential, you'll have to get a copy of the FRED Reference Manual and read it. The Reference Manual is written at a somewhat more advanced level than this tutorial, but now that you're an experienced FRED user, you should have no problem picking up the many features we haven't had time to describe here.

In addition to the Reference Manual, there is also an advanced tutorial that will teach you about writing FRED buffer programs. This manual is available by typing "[expl fred buffers](#)". This advanced tutorial covers a good many commands that we haven't had time to mention here -- anyone who is going to use FRED beyond the elementary level really ought to read the buffer guide.

Whether you're using FRED for simple editing or something fancier, we hope that this guide has made your work easier. Good luck!

Appendix A: ALTERNATE FORMATS FOR CHARACTERS

Some of the characters recognized by FRED are not available on certain terminals. FRED lets you construct alternate forms of these characters with your escape character followed by another character. Below we list the alternate character formats accepted by FRED.

\A	@ (commercial at sign)
\G	(bell)
\H	(back space)
\\	\ (backslash)
\(	{ (left brace)
\)	} (right brace)
\!	(or bar)
\=	~ (tilde or not sign)
_	` (grave accent)
\	<tab> (escape blank)
\;	<carriage return>
\ddd	ASCII character ddd (octal)